

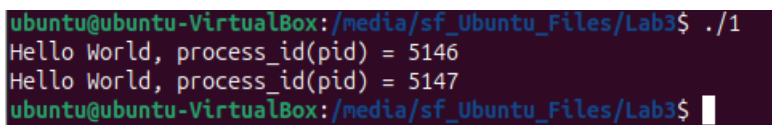
1. Write a program to implement fork () system call.

```
#include <iostream>

#include <unistd.h>

using namespace std;

int main()
{
    pid_t p = fork();
    if( p < 0 )
    {
        perror("Fork Failed!");
        exit(1);
    }
    cout << "Hello World, process_id(pid) = " << getpid() << endl;
    return 0;
}
```



```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab3$ ./1
Hello World, process_id(pid) = 5146
Hello World, process_id(pid) = 5147
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab3$
```

2. Write a program to implement wait () and exit () System Calls.

```
#include <iostream>

#include <sys/wait.h>

#include <unistd.h>

#include <cstdlib>

int main() {
    pid_t child_pid = fork();

    if (child_pid == 0) {
        // Child process
        std::cout << "Child process: Hello from child!" << std::endl;
        exit(42); // Terminate child with exit status 42
    }
}
```

```

} else if (child_pid > 0) {
    // Parent process

    int status;

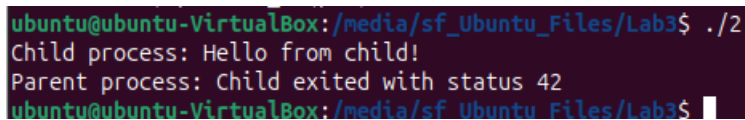
    wait(&status); // Wait for the child process to terminate

    if (WIFEXITED(status)) {
        std::cout << "Parent process: Child exited with status " << WEXITSTATUS(status) << std::endl;
    }
} else {
    std::cerr << "Fork failed!" << std::endl;

    return 1;
}

return 0;
}

```



```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab3$ ./2
Child process: Hello from child!
Parent process: Child exited with status 42
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab3$

```

3. Write a program to implement `execv()` system call.

```
#include <iostream>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    // Path to the executable you want to run
```

```
    const char* path = "/home/ubuntu/OS_Files/Lab_3/2"; // Change this to the actual path
```

```
    // Arguments for the executable (null-terminated array)
```

```
    char* args[] = {const_cast<char*>(path), nullptr};
```

```
    // Execute the program specified by 'path'
```

```
    execv(path, args);
```

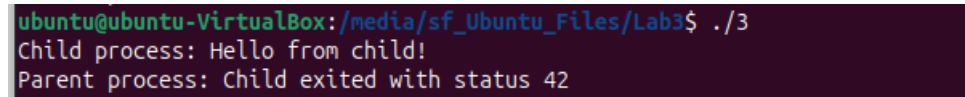
```
    // If execv() fails, this line won't be reached

```

```
std::cerr << "Error executing program\n";

return 1;

}
```



```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab3$ ./3
Child process: Hello from child!
Parent process: Child exited with status 42
```

4. Write a program to implement the system calls open (), read (), write () & close ().

```
#include <iostream>

#include <unistd.h>

#include <fcntl.h> // For open() flags
#include <cstring> // For strerror()

int main() {

    const char* inputFilename = "/home/ubuntu/OS_Files/Lab_3/1.c++"; // Replace with your input file
    const char* outputFilename = "/home/ubuntu/OS_Files/Lab_3/1_.c++"; // Replace with your output file

    // Open the input file for reading
    int inputFileDescriptor = open(inputFilename, O_RDONLY);
    if (inputFileDescriptor == -1) {
        std::cerr << "Error opening " << inputFilename << ": " << strerror(errno) << std::endl;
        return 1;
    }

    // Open the output file for writing (create if not exists)
    int outputFileDescriptor = open(outputFilename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (outputFileDescriptor == -1) {
        std::cerr << "Error opening " << outputFilename << ": " << strerror(errno) << std::endl;
        close(inputFileDescriptor); // Close the input file descriptor
        return 1;
    }

    // Read from input file and write to output file
    char buffer[1024];
```

```

ssize_t bytesRead;
while ((bytesRead = read(inputFileDescriptor, buffer, sizeof(buffer))) > 0) {
    ssize_t bytesWritten = write(outputFileDescriptor, buffer, bytesRead);
    if (bytesWritten == -1) {
        std::cerr << "Error writing to " << outputFilename << ": " << strerror(errno) << std::endl;
        break;
    }
}

// Close both file descriptors
close(inputFileDescriptor);
close(outputFileDescriptor);

std::cout << "Contents copied from " << inputFilename << " to " << outputFilename << std::endl;
return 0;
}

```

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab3$ ./4
Contents copied from /home/ubuntu/OS_Files/Lab_3/1.c++ to /home/ubuntu/OS_Files/
Lab_3/1_.c++

```